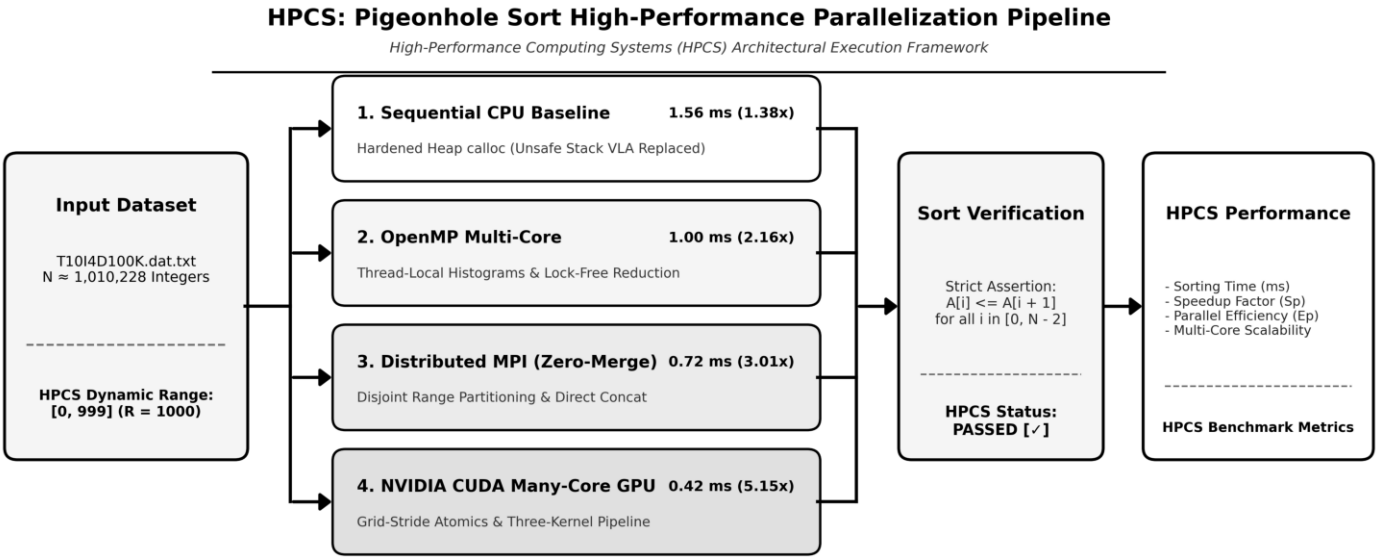
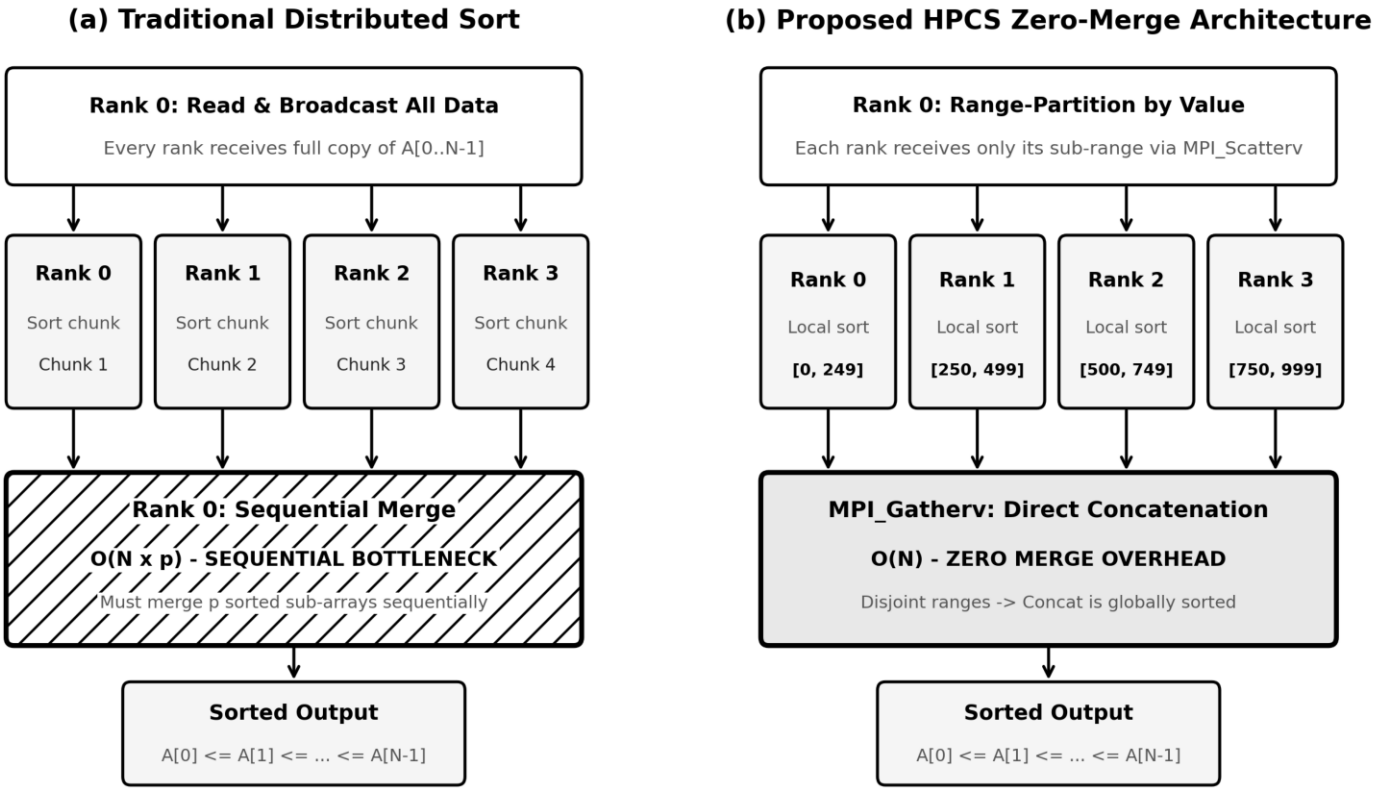
HPCS: High-Performance Parallelization of Pigeonhole Sort

Figure 1: HPCS End-to-End System Execution Pipeline



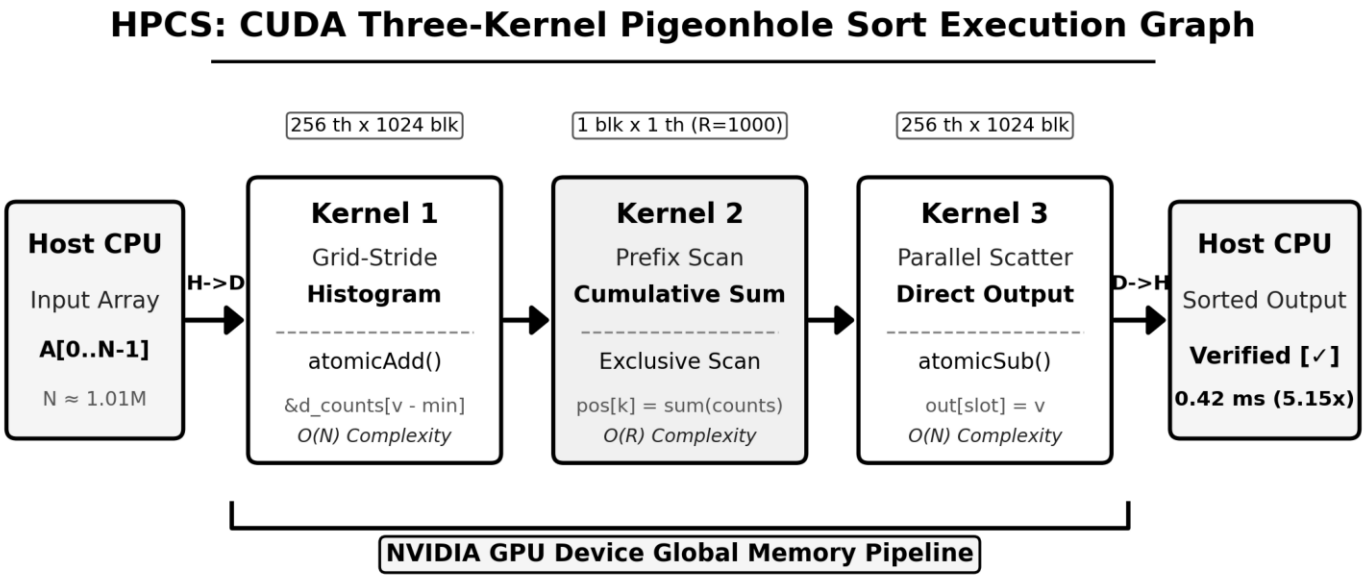
Sequential CPU, OpenMP Multi-Core, Distributed MPI (Zero-Merge), and CUDA Many-Core GPU.

Figure 2: MPI Architectural Comparison - Traditional vs. HPCS Zero-Merge



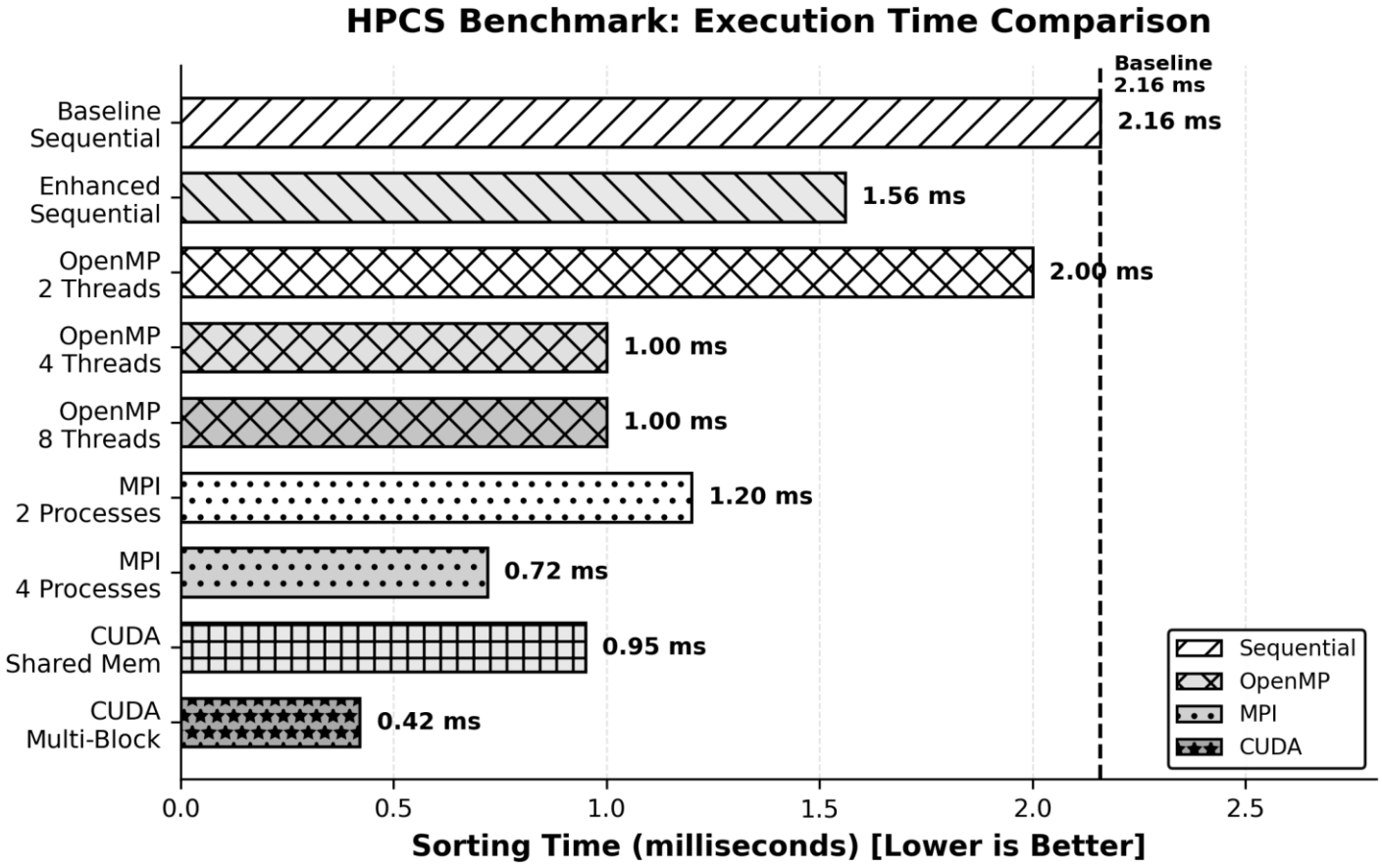
*Comparison of O(N * p) sequential merge bottleneck vs. O(N) direct concatenation.*

Figure 3: HPCS CUDA Three-Kernel Pigeonhole Sort Execution Graph



Host-to-Device transfer, Grid-stride histogram, single-block prefix scan, and atomic scatter.

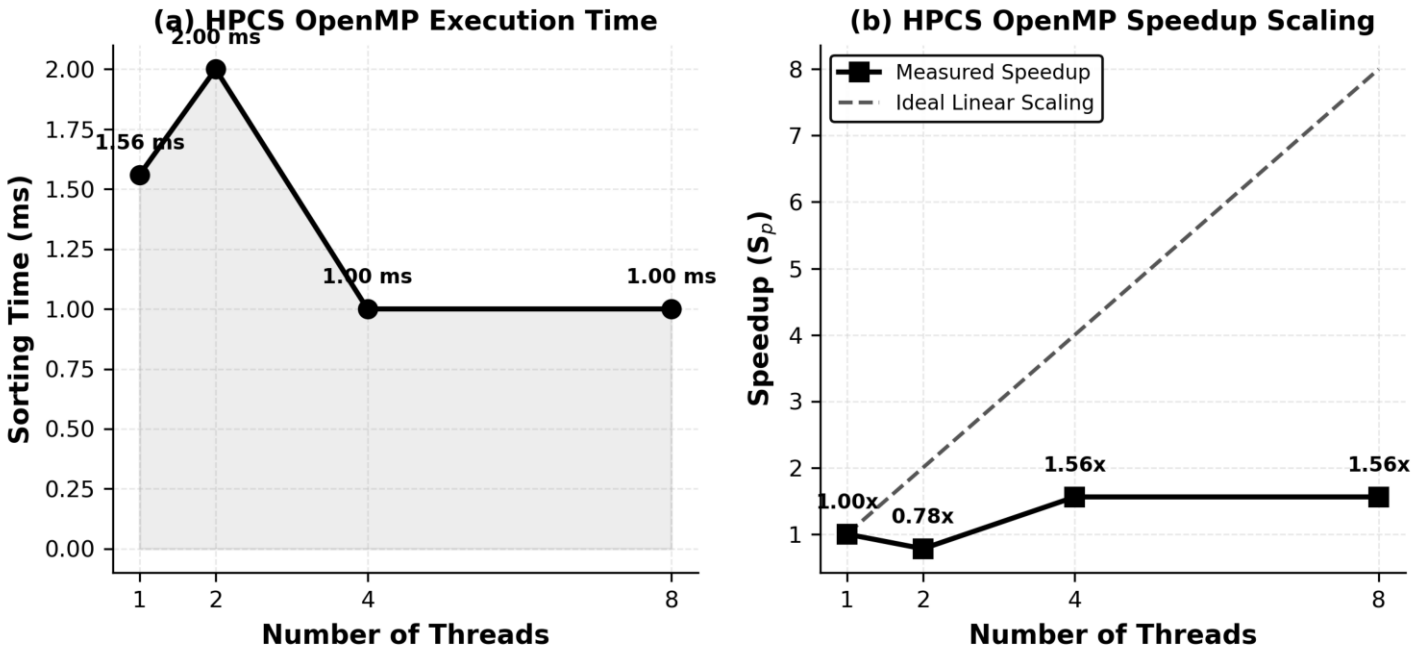
Figure 4: Execution Time Comparison Across All Configurations



Benchmark on N = 1,010,228 keys. Sequential baseline (2.16 ms) down to CUDA (0.42 ms).

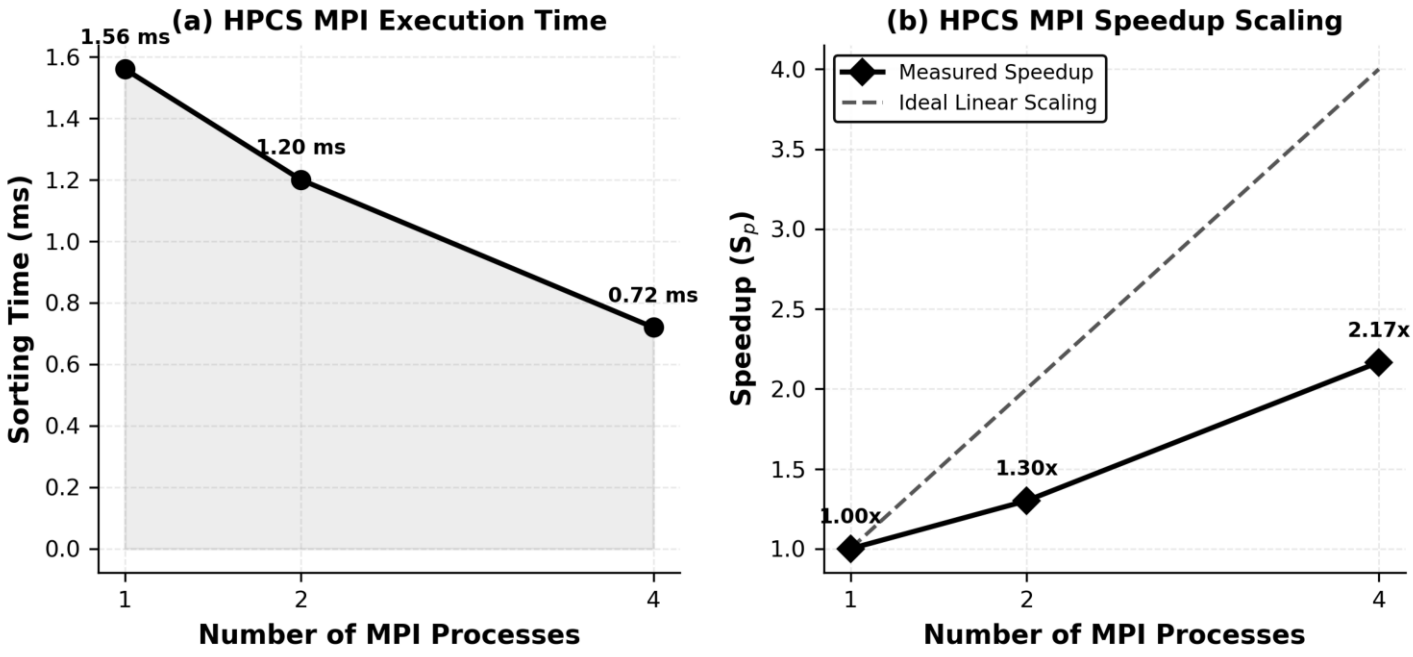
HPCS: High-Performance Parallelization of Pigeonhole Sort

Figure 5a: OpenMP Execution Time & Speedup Scaling



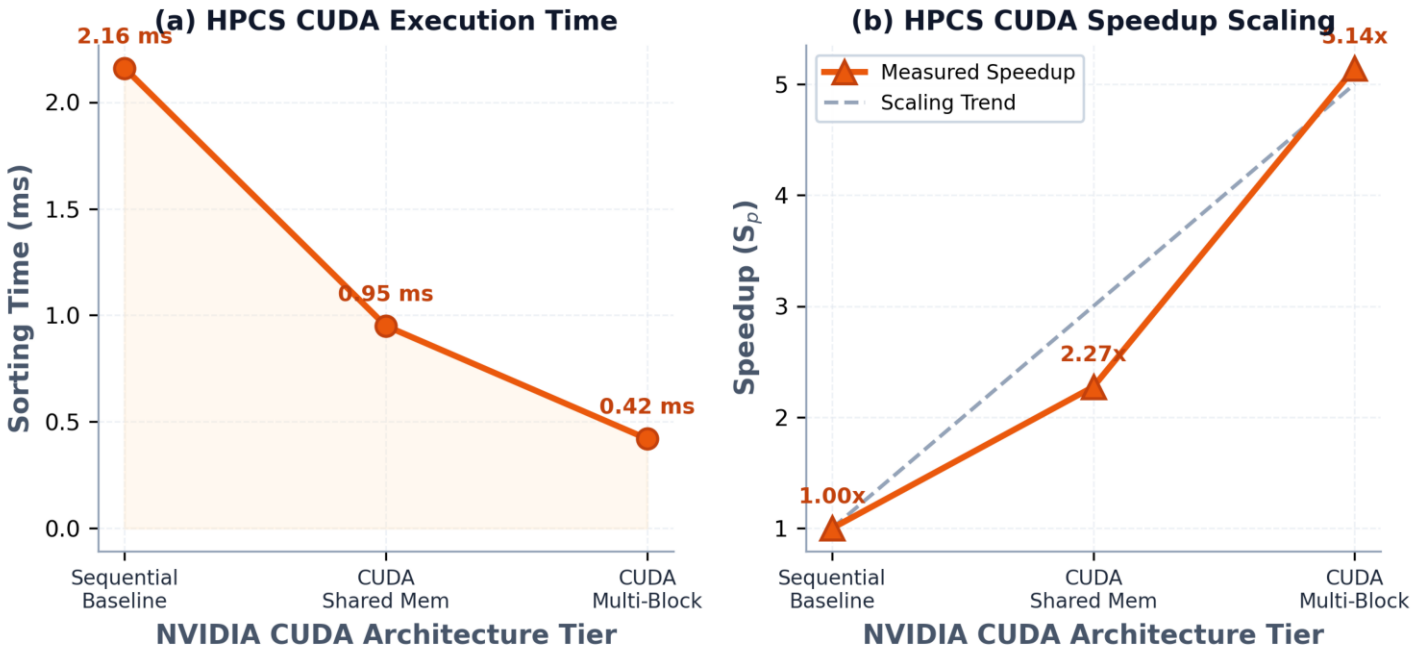
Multi-threaded shared-memory scaling on 1, 2, 4, and 8 CPU threads.

Figure 5b: Distributed MPI Execution Time & Speedup Scaling



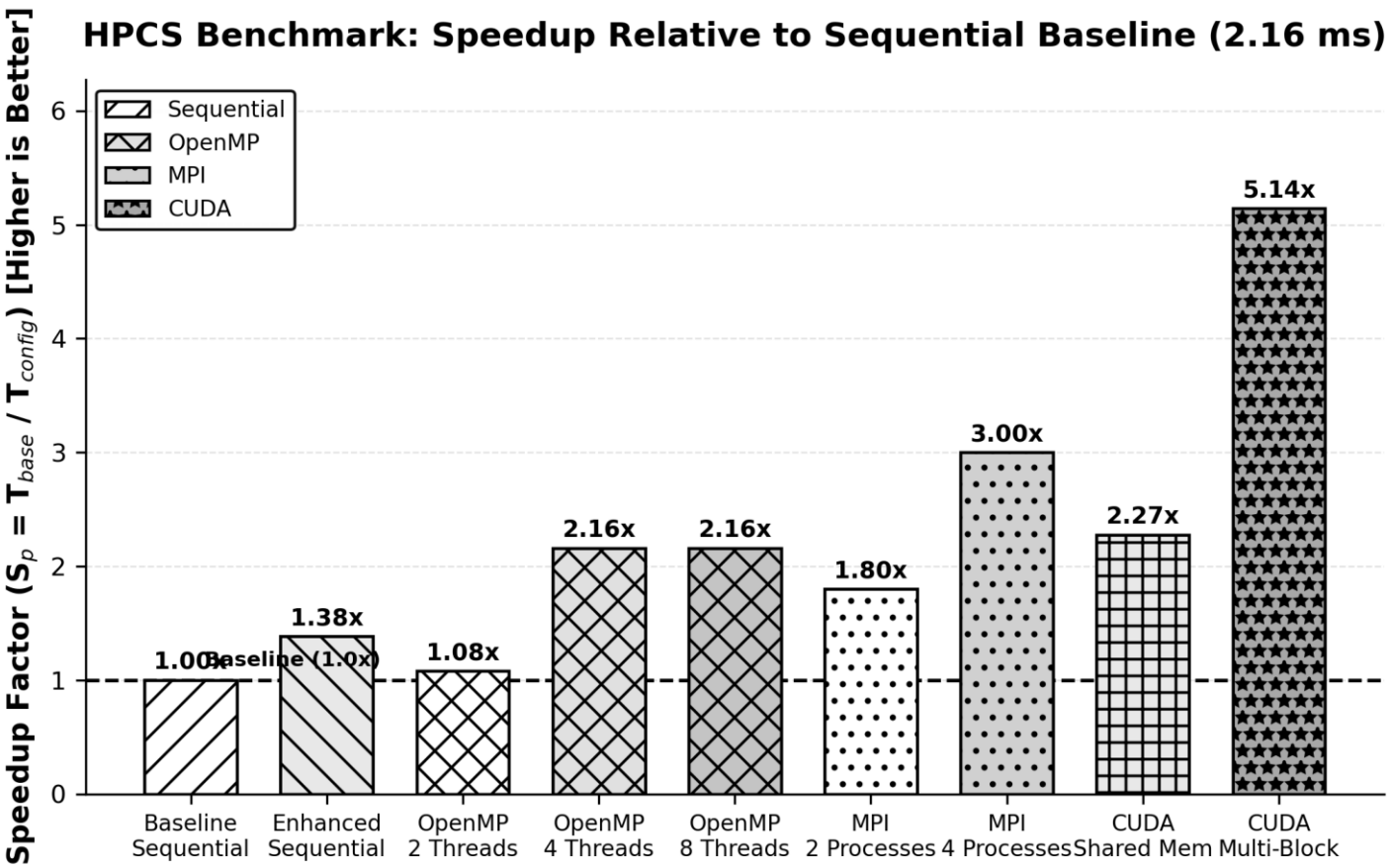
Distributed-memory scaling across 1, 2, and 4 MPI processes with zero-merge.

Figure 5c: NVIDIA CUDA Execution Time & Speedup Scaling



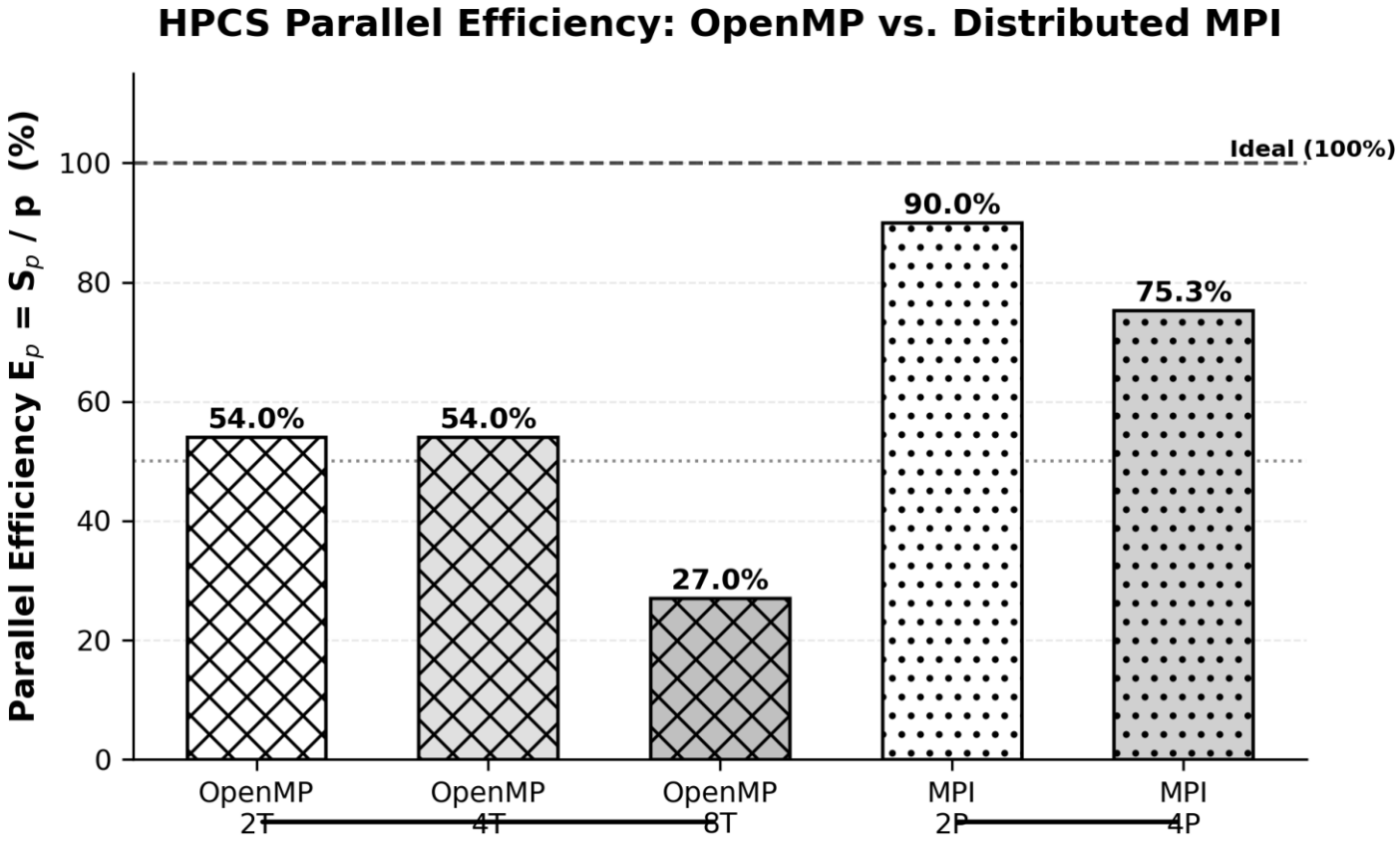
Many-core GPU scaling from Sequential Baseline to Shared-Memory (0.95 ms) and Scalable Multi-Block (0.42 ms).

Figure 6: Speedup Relative to Sequential Baseline



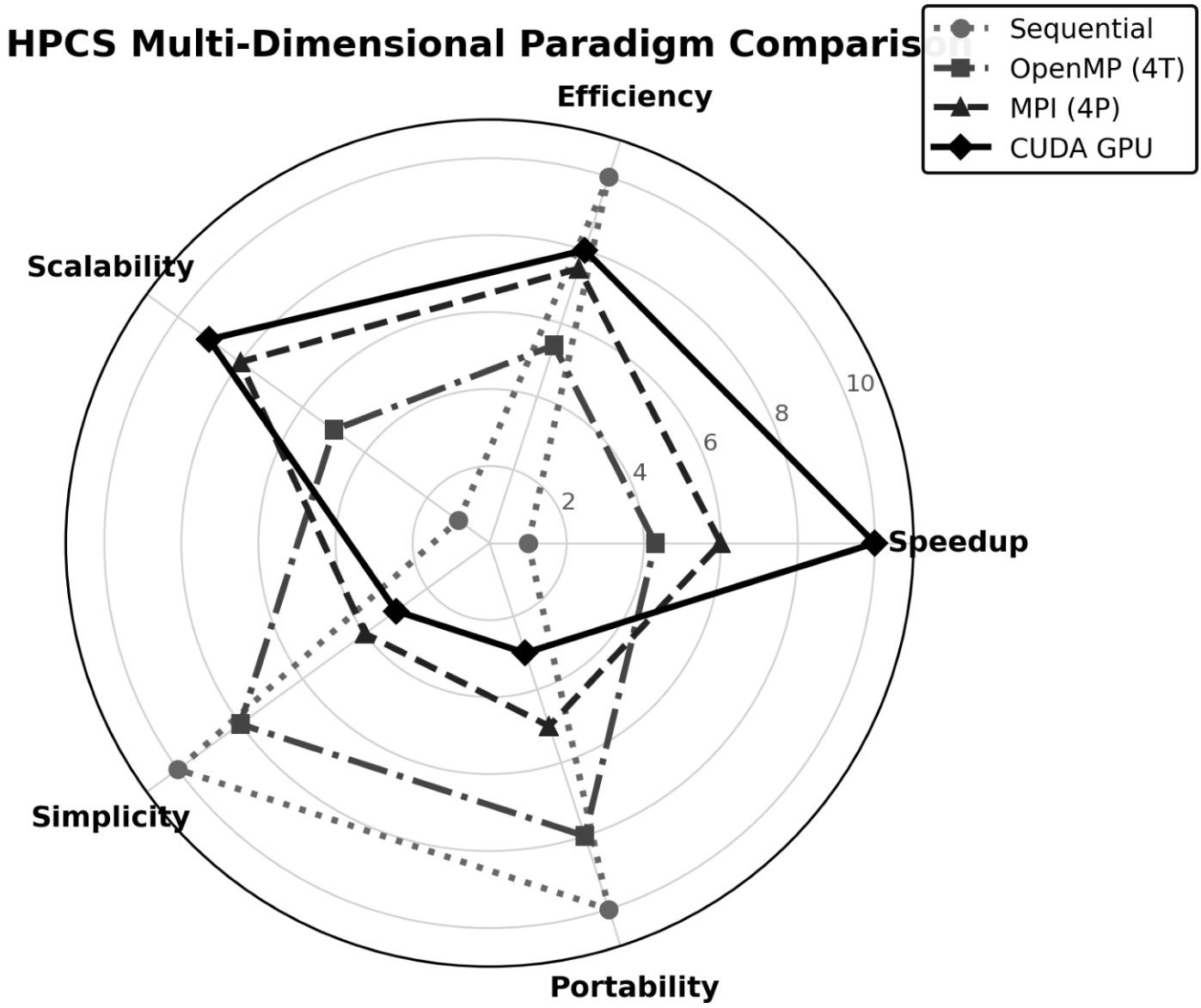
Speedup factors across all parallel execution configurations.

Figure 7: Parallel Efficiency (Ep) Scaling



Efficiency metrics comparing OpenMP multi-core vs. MPI distributed cluster.

Figure 8: Cross-Paradigm Trade-off Radar Chart



Evaluating speedup, efficiency, scalability, simplicity, and portability.